

ExtractArt — CRNN v2 (Final)

HumanAI @ Google Summer of Code 2026

Before running:

1. Runtime → Change runtime type → T4 GPU → Save
2. Run cells top to bottom, one at a time

Two Google Drive accounts:

- Account 1 = CSV files
- Account 2 = images, checkpoints, features (needs free space)

Cell 1 — Check GPU

```
import torch
if torch.cuda.is_available():
    gpu = torch.cuda.get_device_name(0)
    mem = torch.cuda.get_device_properties(0).total_memory / 1e9
    print(f'GPU : {gpu}')
    print(f'VRAM: {mem:.1f} GB')
    print('GPU ready.')
else:
    print('NO GPU — go to Runtime -> Change runtime type -> T4 GPU -> Save')
```

```
GPU : Tesla T4
VRAM: 15.6 GB
GPU ready.
```

Cell 2 — Mount Account 1 (CSV files)

Sign in with the account that has your CSV files.

```
from google.colab import drive
import os
drive.mount('/content/drive', force_remount=True)
CSV_DIR = '/content/drive/MyDrive/ExtractArt/wikiart_csv'
os.makedirs(CSV_DIR, exist_ok=True)
print('Account 1 mounted.')
print(f'CSV folder: {CSV_DIR}')
```

```
Mounted at /content/drive
Account 1 mounted.
CSV folder: /content/drive/MyDrive/ExtractArt/wikiart_csv
```

Cell 3 — Mount Account 2 (Images + Checkpoints)

Sign in with your second Google account that has more free space.

```
from google.colab import drive
import os, shutil
drive.mount('/content/drive', force_remount=True)
BASE2 = '/content/drive/MyDrive/ExtractArt'
IMAGE_DIR = f'{BASE2}/wikiart_images'
CKPT_DIR = f'{BASE2}/checkpoints'
FEATURE_DIR = f'{BASE2}/features'
for d in [BASE2, IMAGE_DIR, CKPT_DIR, FEATURE_DIR]:
    os.makedirs(d, exist_ok=True)
total, used, free = shutil.disk_usage('/content/drive/MyDrive')
print('Account 2 mounted.')
print(f'Total : {total/1e9:.1f} GB')
print(f'Used : {used/1e9:.1f} GB')
print(f'Free : {free/1e9:.1f} GB')
if free/1e9 >= 5:
    print('Enough space for 10,000 images.')
else:
    print(f'Only {free/1e9:.1f} GB free – reduce MAX_IMAGES in Cell 6.')
```

```
Mounted at /content/drive
Account 2 mounted.
Total : 16.1 GB
Used : 3.8 GB
Free : 12.3 GB
Enough space for 10,000 images.
```

Cell 4 — Verify CSV Files

```
import os
CSV_DIR = '/content/drive/MyDrive/ExtractArt/wikiart_csv'
expected = [
    'style_train.csv', 'style_val.csv', 'style_class.txt',
    'genre_train.csv', 'genre_val.csv', 'genre_class.txt',
    'artist_train.csv', 'artist_val.csv', 'artist_class.txt',
]
all_ok = True
for f in expected:
    path = os.path.join(CSV_DIR, f)
    if os.path.exists(path):
        print(f' OK {f} ({os.path.getsize(path):,} bytes)')
    else:
        print(f' MISSING {f}')
        all_ok = False
print()
```

```
print('All CSV files found. Ready to continue.' if all_ok else 'Upload missing files first.')
```

```
OK      style_train.csv  (3,658,221 bytes)
OK      style_val.csv   (1,575,672 bytes)
OK      style_class.txt (462 bytes)
OK      genre_train.csv (2,900,018 bytes)
OK      genre_val.csv   (1,248,086 bytes)
OK      genre_class.txt (156 bytes)
OK      artist_train.csv (865,929 bytes)
OK      artist_val.csv  (370,345 bytes)
OK      artist_class.txt (406 bytes)
```

All CSV files found. Ready to continue.

Cell 5 — Install Dependencies

```
!pip install -q scipy
import torch, torchvision, numpy, pandas, sklearn, PIL, scipy, tqdm, requests
print('torch      : ', torch.__version__)
print('torchvision: ', torchvision.__version__)
print('scipy       : ', scipy.__version__)
print('All dependencies OK.')
```

```
torch      : 2.10.0+cu128
torchvision: 0.25.0+cu128
scipy      : 1.16.3
All dependencies OK.
```

Cell 6 — Test WikiArt URL Connectivity

```
import requests, os
import pandas as pd
from PIL import Image
from io import BytesIO

CSV_DIR = '/content/drive/MyDrive/ExtractArt/wikiart_csv'
HEADERS = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64) Chrome/120.0.0.0',
    'Referer': 'https://www.wikiart.org/',
}

def csv_path_to_url(p):
    name_part = os.path.splitext(os.path.basename(p))[0]
    u = name_part.find('_')
    if u == -1:
        return
```

```

f'https://uploads6.wikiart.org/images/{name_part}/{name_part}.jpg'
    return
f'https://uploads6.wikiart.org/images/{name_part[:u]}/{name_part[u+1:]}
}.jpg'

df = pd.read_csv(os.path.join(CSV_DIR, 'style_train.csv'),
header=None, names=['path', 'label'])
paths = df['path'].str.strip().tolist()[:5]
print('Testing 5 WikiArt URLs...')
ok = 0
for p in paths:
    url = csv_path_to_url(p)
    try:
        r = requests.get(url, headers=HEADERS, timeout=10)
        if r.status_code == 200:
            img = Image.open(BytesIO(r.content))
            print(f' OK      {img.size}  {p}')
            ok += 1
        else:
            print(f' FAIL   {r.status_code}  {p}')
    except Exception as e:
        print(f' ERROR {e}')
print(f'Result: {ok}/5 URLs accessible')

Testing 5 WikiArt URLs...
OK      (986, 631)  Impressionism/edgar-degas_landscape-on-the-
orne.jpg
OK      (3176, 2381)  Realism/camille-corot_mantes-cathedral.jpg
OK      (1400, 1115)  Abstract_Expressionism/gene-davis_untitled-
1979(3).jpg
OK      (811, 1000)  Symbolism/kuzma-petrov-vodkin_in-the-1920.jpg
OK      (855, 1092)  Impressionism/maurice-prendergast_paris-
boulevard-in-the-rain-1893.jpg
Result: 5/5 URLs accessible

```

Cell 7 — Download Images to Account 2

Change MAX_IMAGES before running.

- 3000 = ~2 GB quick test
- 10000 = ~5 GB recommended
- None = full dataset

```

import os, time, requests
import pandas as pd
from PIL import Image
from io import BytesIO
from tqdm.notebook import tqdm

```

```

# — CONFIGURE HERE —————
MAX_IMAGES = 10000
# —————

CSV_DIR = '/content/drive/MyDrive/ExtractArt/wikiart_csv'
IMAGE_DIR = '/content/drive/MyDrive/ExtractArt/wikiart_images'
BASE_URL = 'https://uploads6.wikiart.org/images'
HEADERS = {
    'User-Agent': 'Mozilla/5.0 (Windows NT 10.0; Win64; x64)
    Chrome/120.0.0.0',
    'Accept': 'image/webp,image/apng,image/*,*/*;q=0.8',
    'Referer': 'https://www.wikiart.org/',
}

def csv_path_to_url(p):
    name_part = os.path.splitext(os.path.basename(p))[0]
    u = name_part.find('_')
    if u == -1:
        return f'{BASE_URL}/{name_part}/{name_part}.jpg'
    return f'{BASE_URL}/{name_part[:u]}/{name_part[u+1:]}.jpg'

all_paths = set()
for task in ['style', 'genre', 'artist']:
    for split in ['train', 'val']:
        fp = os.path.join(CSV_DIR, f'{task}_{split}.csv')
        if os.path.exists(fp):
            df = pd.read_csv(fp, header=None, names=['path', 'label'])
            for p in df['path'].str.strip(): all_paths.add(p)

all_paths = sorted(all_paths)
print(f'Total unique images in CSVs: {len(all_paths)}')
if MAX_IMAGES: all_paths = all_paths[:MAX_IMAGES]; print(f'Limited to:
{MAX_IMAGES}')

os.makedirs(IMAGE_DIR, exist_ok=True)
for p in all_paths:
    os.makedirs(os.path.join(IMAGE_DIR, p.split('/')[0]),
    exist_ok=True)

already = sum(1 for p in all_paths if
os.path.exists(os.path.join(IMAGE_DIR, p)))
print(f'Already downloaded: {already}')
print(f'To download          : {len(all_paths) - already}')

session = requests.Session()
saved = already; failed = 0; skipped = 0
pbar = tqdm(all_paths, desc='Downloading')
for rel_path in pbar:
    abs_path = os.path.join(IMAGE_DIR, rel_path)
    if os.path.exists(abs_path): skipped += 1; continue

```

```

    try:
        r = session.get(csv_path_to_url(rel_path), headers=HEADERS,
            timeout=15)
        if r.status_code == 200:
            Image.open(BytesIO(r.content)).convert('RGB').save(abs_path, 'JPEG',
                quality=90)
            saved += 1
        else: failed += 1
    except: failed += 1
    pbar.set_postfix({'saved': saved, 'failed': failed})
    time.sleep(0.05)
session.close()
print(f'Downloaded: {saved} | Failed: {failed} | Skipped: {skipped}')
print(f'Saved to : {IMAGE_DIR}')

Total unique images in CSVs: 81446
Limited to: 10000
Already downloaded: 9758
To download      : 242

{"model_id": "6ba76f5cbd004ade8e29e006bdb603b0", "version_major": 2, "version_minor": 0}

Downloaded: 9758 | Failed: 242 | Skipped: 9758
Saved to : /content/drive/MyDrive/ExtractArt/wikiart_images

```

Cell 8 — Define All Model Classes and Functions

Run every time after a runtime restart.

```

import os, sys, random, warnings
warnings.filterwarnings('ignore')
import numpy as np
import pandas as pd
from PIL import Image
from tqdm import tqdm
from scipy.spatial.distance import mahalanobis as scipy_mahalanobis
import torch
import torch.nn as nn
import torch.nn.functional as F
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from torchvision.models import convnext_base, ConvNeXt_Base_Weights
from sklearn.metrics import f1_score, classification_report
from sklearn.ensemble import IsolationForest

CFG = {
    'epochs': 30, 'batch_size': 8, 'lr': 1e-4, 'weight_decay': 1e-4,

```

```

    'patience': 15, 'dropout': 0.4, 'lstm_hidden': 512, 'lstm_layers':
2,
    'embedding_dim': 512, 'patch_dim': 1024, 'proj_dim': 512,
'n_patches': 49,
    'focal_gamma': 0.5, 'label_smooth': 0.1, 'warmup_T0': 5,
'warmup_Tmult': 2,
    'outlier_conf_thresh': 0.30, 'outlier_sigma': 2.0,
'isolation_contam': 0.05,
    'checkpoint_dir':
'/content/drive2/MyDrive/ExtractArt/checkpoints',
    'feature_dir': '/content/drive2/MyDrive/ExtractArt/features',
    'num_workers': 4, 'seed': 42,
}

def set_seed(seed):
    random.seed(seed); np.random.seed(seed); torch.manual_seed(seed)
    if torch.cuda.is_available(): torch.cuda.manual_seed_all(seed)

def load_class_names(path):
    names = {}
    if not os.path.exists(path): return names
    with open(path) as f:
        for line in f:
            line = line.strip()
            if line:
                idx, name = line.split(' ', 1)
                names[int(idx)] = name
    return names

def load_num_classes(csv_dir, task, fallback):
    txt = os.path.join(csv_dir, f'{task}_class.txt')
    if os.path.exists(txt):
        with open(txt) as f: return sum(1 for line in f if
line.strip())
    return fallback

def discover_tasks(csv_dir):
    tasks = []
    for task in ['style', 'genre', 'artist']:
        if (os.path.exists(os.path.join(csv_dir, f'{task}_train.csv'))
and
            os.path.exists(os.path.join(csv_dir, f'{task}_val.csv'))):
            tasks.append(task)
    if not tasks: raise RuntimeError(f'No task CSVs found in
{csv_dir}')
    return tasks

class WikiArtDataset(Dataset):
    def __init__(self, csv_path, image_dir, transform=None):
        df = pd.read_csv(csv_path, header=None, names=['path',

```

```

'label'])
    df['path'] = df['path'].str.strip()
    df['label'] = df['label'].astype(int)
    df['full'] = df['path'].apply(lambda p:
os.path.join(image_dir, p))
    before = len(df)
    df =
df[df['full'].apply(os.path.exists)].reset_index(drop=True)
    missing = before - len(df)
    if missing: print(f' [Dataset] {os.path.basename(csv_path)}:
skipped {missing}/{before} missing')
    if len(df) == 0: raise RuntimeError('No images found. Check
image_dir path.')
    self.df = df; self.transform = transform
    def __len__(self): return len(self.df)
    def __getitem__(self, idx):
        row = self.df.iloc[idx]
        try: img = Image.open(row['full']).convert('RGB')
        except: img = Image.new('RGB', (224, 224))
        if self.transform: img = self.transform(img)
        return img, int(row['label']), row['path']
    def num_classes(self): return int(self.df['label'].nunique())
    def get_labels(self): return self.df['label'].values.astype(int)

def get_transforms(train):
    mean, std = [0.485, 0.456, 0.406], [0.229, 0.224, 0.225]
    if train:
        return transforms.Compose([
            transforms.Resize(256), transforms.RandomCrop(224),
            transforms.RandomHorizontalFlip(),
            transforms.ColorJitter(0.4, 0.4, 0.3, 0.1),
            transforms.RandomRotation(15),
            transforms.RandomGrayscale(p=0.05),
            transforms.ToTensor(), transforms.Normalize(mean, std),
        ])
    return transforms.Compose([
        transforms.Resize(256), transforms.CenterCrop(224),
        transforms.ToTensor(), transforms.Normalize(mean, std),
    ])

class FocalLossWithSmoothing(nn.Module):
    def __init__(self, n_classes, gamma=2.0, smoothing=0.1,
weight=None):
        super().__init__()
        self.n_classes = n_classes; self.gamma = gamma
        self.smoothing = smoothing; self.weight = weight
    def forward(self, logits, targets):
        with torch.no_grad():
            smooth_val = self.smoothing / (self.n_classes - 1)

```



```

        smooth_target = torch.full_like(logits, smooth_val)
        smooth_target.scatter_(1, targets.unsqueeze(1), 1.0 -
self.smoothing)
        log_prob = F.log_softmax(logits, dim=1)
        if self.weight is not None:
            ce = -(smooth_target * log_prob).sum(dim=1) *
self.weight[targets]
        else:
            ce = -(smooth_target * log_prob).sum(dim=1)
        return ((1.0 - torch.exp(-ce)) ** self.gamma * ce).mean()

class AttentionPool(nn.Module):
    def __init__(self, dim):
        super().__init__()
        self.score = nn.Sequential(nn.Linear(dim, dim//4), nn.Tanh(),
nn.Linear(dim//4, 1))
    def forward(self, x):
        return (torch.softmax(self.score(x), dim=1) * x).sum(dim=1)

class CRNNEExtractor(nn.Module):
    def __init__(self, lstm_hidden=512, lstm_layers=2, dropout=0.4,
embedding_dim=512, patch_dim=1024, proj_dim=512):
        super().__init__()
        backbone =
convnext_base(weights=ConvNext_Base_Weights.DEFAULT)
        self.cnn = backbone.features
        for name, param in self.cnn.named_parameters():
            if int(name.split('.')[0]) <= 0: param.requires_grad =
False
        self.proj = nn.Sequential(
            nn.Linear(patch_dim, proj_dim), nn.LayerNorm(proj_dim),
nn.GELU(),
            nn.Dropout(dropout * 0.5),
            nn.Linear(proj_dim, proj_dim), nn.LayerNorm(proj_dim),
nn.GELU(),
        )
        self.lstm = nn.LSTM(input_size=proj_dim,
hidden_size=lstm_hidden,
num_layers=lstm_layers, batch_first=True,
bidirectional=True,
dropout=dropout if lstm_layers > 1 else 0.0)
        bilstm_dim = lstm_hidden * 2
        self.attention = AttentionPool(bilstm_dim)
        self.embed = nn.Sequential(
            nn.Linear(bilstm_dim, embedding_dim),
nn.LayerNorm(embedding_dim),
            nn.GELU(), nn.Dropout(dropout))
        self.embedding_dim = embedding_dim
    def forward(self, x):

```

```

        B = x.size(0)
        feat = self.cnn(x).view(B, 1024, 49).permute(0, 2, 1)
        feat = self.proj(feat)
        lstm_out, _ = self.lstm(feat)
        return F.normalize(self.embed(self.attention(lstm_out)), p=2,
dim=1)

class MultiTaskCRNN(nn.Module):
    def __init__(self, num_classes, lstm_hidden=512, lstm_layers=2,
        dropout=0.4, embedding_dim=512, patch_dim=1024,
proj_dim=512):
        super().__init__()
        self.extractor = CRNNExtractor(lstm_hidden, lstm_layers,
dropout,
                                embedding_dim, patch_dim,
proj_dim)
        self.heads = nn.ModuleDict({
            task: nn.Linear(embedding_dim, n) for task, n in
num_classes.items()})
        def forward(self, x):
            emb = self.extractor(x)
            return {task: head(emb) for task, head in self.heads.items()},
emb
        @torch.no_grad()
        def embed(self, x): return self.extractor(x)

    def run_epoch(model, loaders, criterions, optimizer, device, train):
        model.train() if train else model.eval()
        ref = max(loaders, key=lambda t: len(loaders[t]))
        n_batches = len(loaders[ref])
        iters = {t: iter(l) for t, l in loaders.items()}
        loss_acc = {t: 0.0 for t in loaders}
        correct = {t: 0 for t in loaders}
        total = {t: 0 for t in loaders}
        all_preds = {t: [] for t in loaders}
        all_tgts = {t: [] for t in loaders}
        all_probs = {t: [] for t in loaders}
        ctx = torch.enable_grad() if train else torch.no_grad()
        with ctx:
            for _ in tqdm(range(n_batches), leave=False, desc='train' if
train else 'val'):
                combined = torch.tensor(0.0, device=device)
                for task, it in iters.items():
                    try: imgs, labels, _ = next(it)
                    except StopIteration:
                        iters[task] = iter(loaders[task]); imgs, labels, _
= next(iters[task])
                    imgs, labels = imgs.to(device), labels.to(device)
                    logits_all, _ = model(imgs)

```

```

        logits = logits_all[task]
        loss = criterions[task](logits, labels)
        combined = combined + loss
        bs = labels.size(0)
        loss_acc[task] += loss.item() * bs
        correct[task] += (logits.argmax(1) ==
labels).sum().item()
        total[task] += bs

all_preds[task].extend(logits.argmax(1).cpu().tolist())
all_tgts[task].extend(labels.cpu().tolist())
all_probs[task].extend(F.softmax(logits,
dim=1).detach().cpu().numpy().tolist())
    if train:
        optimizer.zero_grad(); combined.backward()
        nn.utils.clip_grad_norm_(model.parameters(), 5.0);
optimizer.step()
    metrics = {}
    for task in loaders:
        n = total[task]
        metrics[task] = {
            'loss': loss_acc[task]/n if n else 0.0,
            'acc': correct[task]/n if n else 0.0,
            'f1': f1_score(all_tgts[task], all_preds[task],
average='weighted', zero_division=0),
            'preds': all_preds[task], 'targets': all_tgts[task],
            'probs': np.array(all_probs[task]),
        }
    return metrics

def train_model(model, train_loaders, val_loaders, train_datasets,
cfg, device):
    criterions = {}
    for task in train_loaders:
        n_cls = model.heads[task].out_features
        labels = train_datasets[task].get_labels()
        counts = np.bincount(labels,
minlength=n_cls).astype(np.float32)
        counts = np.where(counts == 0, 1.0, counts)
        weights =
torch.FloatTensor(np.sqrt(1.0/counts/(1.0/counts).sum()*n_cls)).to(dev
ice)

        criterions[task] = FocalLossWithSmoothing(
            n_classes=n_cls, gamma=cfg['focal_gamma'],
            smoothing=cfg['label_smooth'], weight=weights)
        print(f' [{task}] FocalLoss + LabelSmoothing + ClassWeights
({n_cls} classes)')
    optimizer = torch.optim.AdamW(
        filter(lambda p: p.requires_grad, model.parameters()),

```

```

        lr=cfg['lr'], weight_decay=cfg['weight_decay'])
    scheduler = torch.optim.lr_scheduler.CosineAnnealingWarmRestarts(
        optimizer, T_0=cfg['warmup_T0'], T_mult=cfg['warmup_Tmult'],
eta_min=1e-6)
    best_acc = 0.0; best_state = None; patience_ctr = 0
    os.makedirs(cfg['checkpoint_dir'], exist_ok=True)
    ckpt_path = os.path.join(cfg['checkpoint_dir'],
'multitask_best.pth')
    for epoch in range(1, cfg['epochs'] + 1):
        lr = optimizer.param_groups[0]['lr']
        print(f'\n Epoch {epoch}/{cfg["epochs"]} lr={lr:.2e}')
        tr = run_epoch(model, train_loaders, criterions, optimizer,
device, True)
        va = run_epoch(model, val_loaders, criterions, None,
device, False)
        scheduler.step()
        avg_val_acc = np.mean([va[t]['acc'] for t in va])
        for task in tr:
            tl = tr[task]; vl = va[task]
            print(f' [{task:>6}] train loss={tl["loss"]:.4f}
acc={tl["acc"]:.4f} f1={tl["f1"]:.4f} | val loss={vl["loss"]:.4f}
acc={vl["acc"]:.4f} f1={vl["f1"]:.4f}')
            tag = ''
            if avg_val_acc > best_acc:
                best_acc = avg_val_acc
                best_state = {k: v.clone() for k, v in
model.state_dict().items()}
                patience_ctr = 0; tag = ' <- best'
                torch.save({'model_state_dict': best_state, 'epoch':
epoch, 'val_acc': best_acc}, ckpt_path)
            else:
                patience_ctr += 1
                if patience_ctr >= cfg['patience']:
                    print(f' Early stopping at epoch {epoch}.'); break
                print(f' Avg val acc: {avg_val_acc:.4f}{tag}')
        print(f'\n Best avg val acc: {best_acc:.4f} -> {ckpt_path}')
        model.load_state_dict(best_state)
    return model

def evaluate(model, val_loaders, device, class_names):
    model.eval()
    print('\n' + '='*68)
    print(' EVALUATION REPORT')
    print('='*68)
    results = {}
    for task, loader in val_loaders.items():
        all_preds, all_tgts, all_probs, top5_hits = [], [], [], []
        with torch.no_grad():
            for imgs, labels, _ in tqdm(loader, desc=f'Eval [{task}]',

```

```

leave=False):
    imgs, labels = imgs.to(device), labels.to(device)
    logits_all, _ = model(imgs)
    logits = logits_all[task]
    probs = F.softmax(logits, dim=1)
    k = min(5, logits.size(1))
    top5 = logits.topk(k, dim=1).indices
    all_preds.extend(logits.argmax(1).cpu().tolist())
    all_tgts.extend(labels.cpu().tolist())

all_probs.extend(probs.detach().cpu().numpy().tolist())
top5_hits.extend([labels[i].item() in top5[i].tolist()
for i in range(len(labels))])
preds_np = np.array(all_preds); tgts_np = np.array(all_tgts)
probs_np = np.array(all_probs)
top1 = (preds_np == tgts_np).mean()
top5a = np.mean(top5_hits)
wf1 = f1_score(tgts_np, preds_np, average='weighted',
zero_division=0)
mf1 = f1_score(tgts_np, preds_np, average='macro',
zero_division=0)
present = sorted(set(all_tgts))
names = [class_names[task].get(i, str(i)) for i in present]
level = 'GENERAL' if task in ('style', 'genre') else
'SPECIFIC'
print(f'\n Task: {task.upper()} [{level}] ({len(present)}
classes present)')
print(f' Top-1 Accuracy : {top1:.4f}')
print(f' Top-5 Accuracy : {top5a:.4f}')
print(f' Weighted F1-Score : {wf1:.4f}')
print(f' Macro F1-Score : {mf1:.4f}')
print(classification_report(tgts_np, preds_np, labels=present,
target_names=names, zero_division=0))
results[task] = {'top1': top1, 'top5': top5a, 'wf1': wf1,
'mf1': mf1,
'preds': all_preds, 'targets': all_tgts,
'probs': probs_np}
return results

def extract_features(model, val_loaders, device, cfg):
    model.eval()
    os.makedirs(cfg['feature_dir'], exist_ok=True)
    store = {}
    for task, loader in val_loaders.items():
        embs, labels, paths = [], [], []
        with torch.no_grad():
            for imgs, lbls, pths in tqdm(loader, desc=f'Features
[{task}]'):
                e = model.embed(imgs.to(device))

```

```

        embs.append(e.cpu().numpy());
labels.extend(lbls.tolist()); paths.extend(pths)
        embs_np = np.concatenate(embs, axis=0).astype(np.float32)
        labels_np = np.array(labels, dtype=np.int32)
        np.save(os.path.join(cfg['feature_dir'],
f'{task}_embeddings.npy'), embs_np)
        np.save(os.path.join(cfg['feature_dir'],
f'{task}_labels.npy'), labels_np)
        np.save(os.path.join(cfg['feature_dir'], f'{task}_paths.npy'),
np.array(paths, dtype=object))
        store[task] = {'embeddings': embs_np, 'labels': labels_np,
'paths': paths}
        print(f' [{task}] {embs_np.shape} saved to Drive')
    return store

def compute_mahalanobis(embeddings_class):
    n, d = embeddings_class.shape
    if n < 3:
        centroid = embeddings_class.mean(axis=0)
        return np.linalg.norm(embeddings_class - centroid, axis=1)
    centroid = embeddings_class.mean(axis=0)
    cov = np.cov(embeddings_class.T) + np.eye(d) * 1e-6
    inv_cov = np.linalg.pinv(cov)
    return np.array([scipy_mahalanobis(e, centroid, inv_cov) for e in
embeddings_class])

def detect_outliers(embeddings, labels, paths, probs, class_names,
task, cfg, n_show=20):
    print(f'\n OUTLIER DETECTION -- {task.upper()}')
    print(' ' + '='*60)
    n = len(embeddings); flags = np.zeros(n, dtype=bool)
    nm = lambda l: class_names.get(l, str(l))
    max_conf = probs.max(axis=1)
    low_conf = max_conf < cfg['outlier_conf_thresh']
    flags |= low_conf
    print(f' A) Low-confidence (<{cfg["outlier_conf_thresh"]}):
{low_conf.sum()} / {n}')
    maha_dists = np.zeros(n)
    for cid in np.unique(labels):
        mask = labels == cid
        maha_dists[mask] = compute_mahalanobis(embeddings[mask])
    thr = maha_dists.mean() + cfg['outlier_sigma'] * maha_dists.std()
    far_ctr = maha_dists > thr
    flags |= far_ctr
    print(f' B) Mahalanobis dist > {thr:.4f}: {far_ctr.sum()} / {n}')
    iso = IsolationForest(n_estimators=200,
contamination=cfg['isolation_contam'], random_state=cfg['seed'])
    iso_flag = iso.fit_predict(embeddings) == -1
    flags |= iso_flag

```

```

print(f' C) Isolation Forest: {iso_flag.sum()} / {n}')
print(f' Combined: {flags.sum()} / {n} flagged')
idxs = np.where(flags)[0]
sorted_ = idxs[np.argsort(maha_dists[idxs])[:, -1]][:n_show]
df = pd.DataFrame({
    'path': [paths[i] for i in sorted_],
    'assigned_class': [nm(labels[i]) for i in sorted_],
    'max_confidence': np.round(max_conf[sorted_], 4),
    'mahalanobis_dist': np.round(maha_dists[sorted_], 4),
    'signal_A': low_conf[sorted_], 'signal_B': far_ctr[sorted_],
    'signal_C': iso_flag[sorted_],
    'n_signals': low_conf[sorted_].astype(int) +
    far_ctr[sorted_].astype(int) + iso_flag[sorted_].astype(int),
}).sort_values(['n_signals', 'mahalanobis_dist'],
    ascending=[False, False]).reset_index(drop=True)
out = os.path.join(cfg['checkpoint_dir'], f'outliers_{task}.csv')
df.to_csv(out, index=False)
print(f' Outlier report saved -> {out}')
return df

print('All classes and functions defined successfully.')
print('Ready to train.')

```

All classes and functions defined successfully.
Ready to train.

Cell 9 — Train the Model

Change EPOCHS and BATCH_SIZE before running.

- Use EPOCHS = 2 first to confirm no errors, then set to 25
- Keep BATCH_SIZE = 8 to avoid OOM errors on T4

```

import os, gc, torch
os.environ['PYTORCH_ALLOC_CONF'] = 'expandable_segments:True'
gc.collect(); torch.cuda.empty_cache()
free, total = torch.cuda.mem_get_info()
print(f'GPU memory free : {free/1e9:.2f} GB')
print(f'GPU memory total: {total/1e9:.2f} GB')

# — CONFIGURE HERE —
EPOCHS      = 5    # Use 2 first to confirm no errors, then 25
BATCH_SIZE  = 8    # Keep at 8 for T4 with ConvNeXt-Base
#

CSV_DIR     = '/content/drive/MyDrive/ExtractArt/wikiart_csv'
IMAGE_DIR   = '/content/drive/MyDrive/ExtractArt/wikiart_images'
CFG['epochs'] = EPOCHS; CFG['batch_size'] = BATCH_SIZE
set_seed(CFG['seed'])
device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')

```

```

print(f'Device      : {device}')
print(f'Epochs     : {EPOCHS}')
print(f'Batch size  : {BATCH_SIZE}')
print(f'Patience    : {CFG["patience"]}')

tasks = discover_tasks(CSV_DIR)
print(f'Tasks: {tasks}')
class_names = {task: load_class_names(os.path.join(CSV_DIR,
f'{task}_class.txt')) for task in tasks}

num_classes = {}; train_loaders = {}; val_loaders = {}; train_datasets
= {}
loader_kw = dict(batch_size=BATCH_SIZE, num_workers=4,
pin_memory=True)
print('\nLoading datasets...')
for task in tasks:
    tr = WikiArtDataset(os.path.join(CSV_DIR, f'{task}_train.csv'),
IMAGE_DIR, get_transforms(True))
    va = WikiArtDataset(os.path.join(CSV_DIR, f'{task}_val.csv'),
IMAGE_DIR, get_transforms(False))
    n_cls = load_num_classes(CSV_DIR, task, tr.num_classes())
    level = '[GENERAL]' if task in ('style', 'genre') else
'[SPECIFIC]'
    print(f' {level} {task:>6}: train={len(tr):>6} val={len(va):>6}
classes={n_cls}')
    num_classes[task] = n_cls; train_datasets[task] = tr
    train_loaders[task] = DataLoader(tr, shuffle=True, **loader_kw)
    val_loaders[task] = DataLoader(va, shuffle=False, **loader_kw)

model = MultiTaskCRNN(
    num_classes=num_classes, lstm_hidden=CFG['lstm_hidden'],
    lstm_layers=CFG['lstm_layers'], dropout=CFG['dropout'],
    embedding_dim=CFG['embedding_dim'], patch_dim=CFG['patch_dim'],
    proj_dim=CFG['proj_dim']).to(device)

tp = sum(p.numel() for p in model.parameters() if p.requires_grad)
print(f'\nTrainable parameters: {tp:,}')
print('\nStarting training...')
model = train_model(model, train_loaders, val_loaders, train_datasets,
CFG, device)
print('\nTraining complete!')

GPU memory free : 15.53 GB
GPU memory total: 15.64 GB
Device      : cuda
Epochs     : 5
Batch size  : 8
Patience   : 15
Tasks: ['style', 'genre', 'artist']

```


Loading datasets...

```
[Dataset] style_train.csv: skipped 50185/57025 missing
[Dataset] style_val.csv: skipped 21503/24421 missing
[GENERAL ] style: train= 6840 val= 2918 classes=27
[Dataset] genre_train.csv: skipped 40584/45503 missing
[Dataset] genre_val.csv: skipped 17366/19492 missing
[GENERAL ] genre: train= 4919 val= 2126 classes=10
[Dataset] artist_train.csv: skipped 12412/13346 missing
[Dataset] artist_val.csv: skipped 5321/5706 missing
[SPECIFIC] artist: train= 934 val= 385 classes=23
```

Downloading: "https://download.pytorch.org/models/convnext_base-6075fbad.pth" to /root/.cache/torch/hub/checkpoints/convnext_base-6075fbad.pth

100%|██████████| 338M/338M [00:01<00:00, 205MB/s]

Trainable parameters: 99,668,797

Starting training...

```
[style] FocalLoss + LabelSmoothing + ClassWeights (27 classes)
[genre] FocalLoss + LabelSmoothing + ClassWeights (10 classes)
[artist] FocalLoss + LabelSmoothing + ClassWeights (23 classes)
```

Epoch 1/5 lr=1.00e-04

```
[ style] train loss=0.0216 acc=0.6757 f1=0.6716 | val loss=0.0167
acc=0.7790 f1=0.7724
[ genre] train loss=1.0176 acc=0.6236 f1=0.6029 | val loss=0.9662
acc=0.6737 f1=0.6702
[artist] train loss=0.1066 acc=0.5456 f1=0.5131 | val loss=0.1133
acc=0.6604 f1=0.5881
Avg val acc: 0.7044 <- best
```

Epoch 2/5 lr=9.05e-05

```
[ style] train loss=0.0159 acc=0.7468 f1=0.7403 | val loss=0.0125
acc=0.7570 f1=0.7555
[ genre] train loss=0.8262 acc=0.7485 f1=0.7400 | val loss=0.8290
acc=0.7080 f1=0.6921
[artist] train loss=0.0859 acc=0.6181 f1=0.5710 | val loss=0.0955
acc=0.7416 f1=0.7125
Avg val acc: 0.7355 <- best
```

Epoch 3/5 lr=6.58e-05

```

[ style] train loss=0.0134 acc=0.7772 f1=0.7740 | val loss=0.0116
acc=0.7464 f1=0.7476
[ genre] train loss=0.6900 acc=0.8194 f1=0.8157 | val loss=0.7137
acc=0.7467 f1=0.7446
[artist] train loss=0.0746 acc=0.6537 f1=0.6203 | val loss=0.0938
acc=0.7224 f1=0.6866
Avg val acc: 0.7385 <- best

```

Epoch 4/5 lr=3.52e-05

```

[ style] train loss=0.0123 acc=0.7930 f1=0.7902 | val loss=0.0107
acc=0.7875 f1=0.7853
[ genre] train loss=0.6064 acc=0.8671 f1=0.8650 | val loss=0.6853
acc=0.7820 f1=0.7795
[artist] train loss=0.0695 acc=0.6875 f1=0.6601 | val loss=0.0875
acc=0.7388 f1=0.6982
Avg val acc: 0.7694 <- best

```

Epoch 5/5 lr=1.05e-05

```

[ style] train loss=0.0120 acc=0.7943 f1=0.7919 | val loss=0.0105
acc=0.7608 f1=0.7600
[ genre] train loss=0.5384 acc=0.9057 f1=0.9043 | val loss=0.6509
acc=0.7975 f1=0.7935
[artist] train loss=0.0675 acc=0.7206 f1=0.6955 | val loss=0.0854
acc=0.7290 f1=0.6936
Avg val acc: 0.7624

```

```

Best avg val acc: 0.7694 ->
/content/drive2/MyDrive/ExtractArt/checkpoints/multitask_best.pth

```

Training complete!

Cell 10 — Evaluate

Run immediately after Cell 9 finishes.

```

print('Running evaluation...')
eval_res = evaluate(model, val_loaders, device, class_names)
print('\n' + '='*68)
print(' SUMMARY')
print('='*68)
print(f" {'Task':<8} {'Level':<10} {'Top-1':>6} {'Top-5':>6} {'W-
F1':>6} {'M-F1':>6}")
print(' ' + '-'*60)
for task in tasks:

```

```

r = eval_res[task]
level = 'GENERAL' if task in ('style', 'genre') else 'SPECIFIC'
print(f' {task:<8} {level:<10} {r["top1"]:>6.4f}
{r["top5"]:>6.4f} {r["wfl"]:>6.4f} {r["mfl"]:>6.4f}')
print('='*68)

```

Running evaluation...

EVALUATION REPORT

Task: STYLE [GENERAL] (5 classes present)

Top-1 Accuracy : 0.7875

Top-5 Accuracy : 1.0000

Weighted F1-Score : 0.7853

Macro F1-Score : 0.6118

	precision	recall	f1-score	support
Abstract_Expressionism	0.92	0.95	0.93	826
Action_painting	0.00	0.00	0.00	25
Analytical_Cubism	0.54	0.88	0.67	33
Art_Nouveau	0.83	0.70	0.76	1270
Baroque	0.64	0.78	0.70	764
accuracy			0.79	2918
macro avg	0.58	0.66	0.61	2918
weighted avg	0.79	0.79	0.79	2918

Task: GENRE [GENERAL] (10 classes present)

Top-1 Accuracy : 0.8170

Top-5 Accuracy : 0.9793

Weighted F1-Score : 0.8133

Macro F1-Score : 0.7296

	precision	recall	f1-score	support
abstract_painting	0.94	0.98	0.96	687
cityscape	0.81	0.71	0.76	76
genre_painting	0.71	0.68	0.70	358
illustration	0.72	0.85	0.78	198
landscape	0.76	0.76	0.76	93
nude_painting	0.63	0.45	0.52	38
portrait	0.79	0.85	0.82	353
religious_painting	0.88	0.70	0.78	202

sketch_and_study	0.73	0.47	0.57	68	
still_life	0.63	0.68	0.65	53	
accuracy			0.82	2126	
macro avg	0.76	0.71	0.73	2126	
weighted avg	0.82	0.82	0.81	2126	
Task: ARTIST [SPECIFIC] (7 classes present)					
Top-1 Accuracy	: 0.7247				
Top-5 Accuracy	: 0.9948				
Weighted F1-Score	: 0.6836				
Macro F1-Score	: 0.4881				
	precision	recall	f1-score	support	
Boris_Kustodiev	0.64	0.14	0.23	63	
Martiros_Saryan	0.00	0.00	0.00	2	
Nicholas_Roerich	0.81	0.69	0.74	138	
Pablo_Picasso	1.00	0.83	0.91	18	
Pyotr_Konchalovsky	0.00	0.00	0.00	2	
Raphael_Kirchner	0.68	0.99	0.80	154	
Salvador_Dali	0.57	1.00	0.73	8	
accuracy			0.72	385	
macro avg	0.53	0.52	0.49	385	
weighted avg	0.72	0.72	0.68	385	
=====					
SUMMARY					
=====					
Task	Level	Top-1	Top-5	W-F1	M-F1

style	GENERAL	0.7875	1.0000	0.7853	0.6118
genre	GENERAL	0.8170	0.9793	0.8133	0.7296
artist	SPECIFIC	0.7247	0.9948	0.6836	0.4881
=====					

Cell 11 — Extract Embeddings + Find Outliers

Run immediately after Cell 10 finishes.

```
print('Extracting embeddings...')
store = extract_features(model, val_loaders, device, CFG)
print('\nRunning outlier detection...')
```

```

outlier_dfs = {}
for task in tasks:
    fs = store[task]
    if len(fs['embeddings']) == 0: continue
    outlier_dfs[task] = detect_outliers(
        fs['embeddings'], fs['labels'], fs['paths'],
        eval_res[task]['probs'], class_names[task], task, CFG)
print('\n' + '='*68)
print('  FINAL SUMMARY')
print('='*68)
print(f"  {'Task':<8}  {'Level':<10}  {'Top-1':>6}  {'Top-5':>6}  {'W-
F1':>6}  {'M-F1':>6}  {'Outliers':>9}")
print('  ' + '-'*68)
for task in tasks:
    r = eval_res[task]
    level = 'GENERAL' if task in ('style', 'genre') else 'SPECIFIC'
    n_out = len(outlier_dfs.get(task, []))
    print(f'  {task:<8}  {level:<10}  {r["top1"]:>6.4f}
{r["top5"]:>6.4f}  {r["wfl"]:>6.4f}  {r["mfl"]:>6.4f}  {n_out:>9}')
print('='*68)
print('\nAll outputs saved to Google Drive Account 2:')
print(f'  Checkpoint : {CFG["checkpoint_dir"]}/multitask_best.pth')
print(f'  Embeddings  : {CFG["feature_dir"]}/')
print(f'  Outliers    : {CFG["checkpoint_dir"]}/outliers_*.csv')

```

Extracting embeddings...

Features [style]: 100%|██████████| 365/365 [01:14<00:00, 4.92it/s]

[style] (2918, 512) saved to Drive

Features [genre]: 100%|██████████| 266/266 [00:53<00:00, 4.97it/s]

[genre] (2126, 512) saved to Drive

Features [artist]: 100%|██████████| 49/49 [00:05<00:00, 8.17it/s]

[artist] (385, 512) saved to Drive

Running outlier detection...

OUTLIER DETECTION -- STYLE

=====

A) Low-confidence (<0.3): 1284 / 2918

B) Mahalanobis dist > 24.5977: 127 / 2918

C) Isolation Forest: 146 / 2918

Combined: 1341 / 2918 flagged

Outlier report saved ->

/content/drive2/MyDrive/ExtractArt/checkpoints/outliers_style.csv

OUTLIER DETECTION -- GENRE

```
=====
A) Low-confidence (<0.3): 137 / 2126
B) Mahalanobis dist > 17.0515: 104 / 2126
C) Isolation Forest: 107 / 2126
Combined: 247 / 2126 flagged
Outlier report saved ->
/content/drive2/MyDrive/ExtractArt/checkpoints/outliers_genre.csv
```

```
=====
OUTLIER DETECTION -- ARTIST
=====
A) Low-confidence (<0.3): 227 / 385
B) Mahalanobis dist > 13.9908: 0 / 385
C) Isolation Forest: 20 / 385
Combined: 227 / 385 flagged
Outlier report saved ->
/content/drive2/MyDrive/ExtractArt/checkpoints/outliers_artist.csv
```

```
=====
FINAL SUMMARY
=====
```

Task	Level	Top-1	Top-5	W-F1	M-F1	Outliers
style	GENERAL	0.7875	1.0000	0.7853	0.6118	20
genre	GENERAL	0.8170	0.9793	0.8133	0.7296	20
artist	SPECIFIC	0.7247	0.9948	0.6836	0.4881	20

```
=====
```

```
All outputs saved to Google Drive Account 2:
Checkpoint :
/content/drive2/MyDrive/ExtractArt/checkpoints/multitask_best.pth
Embeddings : /content/drive2/MyDrive/ExtractArt/features/
Outliers   :
/content/drive2/MyDrive/ExtractArt/checkpoints/outliers_*.csv
```